Specifying What Not to Do: Quality Beyond "It Builds and Lintian Is Quiet" in LLM-Driven Debian Autopackaging

Andrew Dougherty*

September 2026

Abstract

The first version of the FRDCSA autopackager drives a language model in a generate–build–revise loop and accepts a source package once `dpkg-buildpackage` and `lintian` both exit cleanly. That loop now produces packages, and a first review of six of them shows that it also produces a characteristic class of defect: artifacts that are well formed and pass the acceptance test while failing to do what they claim to do. We describe four such defects, argue that each is an instance of the loop optimising its own acceptance predicate rather than packaging quality, and propose a layered set of remedies. The central proposal is a *negative specification*—a lintian-like rule base, evaluated outside the model, stating what an autopackaged artifact must never contain or do—combined with behavioural oracles, a structured intermediate analysis that turns hollow packages into detectable contradictions, and disciplined revision. We also assess where chain-of-thought, graph-of-thought and backtracking search are likely to help, and conclude that they are secondary: the binding constraint is the verifier, not the generator. A first implementation of the negative specification, in SWI-Prolog, catches the override defect mechanically, and its known-good corpus has already exposed false positives in its own rules.

1 The naive model

The current pipeline can be stated in three lines. Given an upstream tree U, a model proposes a `debian/` directory D_0. For $i = 0, 1, \ldots$ the pipeline builds $U + D_i$ in an isolated VM, runs lintian, and if either reports an error, feeds the log back to the model to obtain D_{i+1}. It stops when

$$Accept(D) \;\equiv\; builds(U + D) \wedge lintian_clean(U + D). \tag{1}$$

This was the right first oracle: it is cheap, fully mechanical, and it forced the loop to work end to end. It has packaged, among others, `eprover`, `souffle`, `VAL`, `fast-downward` and `leo-iii` for a separate unofficial archive. The question this note addresses is what to do now that it works.

2 Observed failure mechanisms

A review of six generated `debian/` directories found the following. The review was carried out by a language model (Claude, Anthropic) at the author's request, to prepare the packages for an

*FRDCSA project, `https://github.com/aindilis/autopackager`; the implementation described in §10 is at `https://github.com/aindilis/antilint`. This note was drafted with the assistance of Claude (Anthropic) from the author's notes and review output; the author has reviewed it and is responsible for its content.

outside Debian developer; the author checked its findings, but they have not yet been confirmed by a Debian developer. None were caught by (1); all were found by reading the output rather than by any mechanical check.

F1: The no-op patch (`souffle`).
The patch `0001-disable-git-version-detection.patch` in `debian/patches/` carries a complete DEP-3 header describing a change, and a diff body that changes nothing. The `series` file is empty, so it is not applied either. The artifact has the *form* of a fix without the substance.

F2: Suppression instead of repair (`val`).
`debian/val.lintian-overrides` silences `no-changelog-file`, with a justifying comment that concerns something else. Because lintian cleanliness is half of the oracle, an override is a direct route to acceptance that bypasses the defect.

F3: Network at build time, undeclared bundling (`leo-iii`).
`debian/rules` downloads `sbt-launch.jar` from Maven during the build and assembles a fat jar whose bundled dependencies are absent from `debian/copyright`. It built, which is all the loop asked.

F4: The hollow install (`fast-downward`).
The install step is guarded so that it silently ships nothing if neither of the expected binary paths exists. A package that installs no program still builds and may still lint cleanly.

Two further observations are about the corpus rather than a package: a trivial package (`vampire`: one CMake call, one install line) tells us little about whether the approach generalises, and an unfinished one (`ape`) builds and installs without being shown to function. Any evaluation should weight packages by difficulty and must not count "installs" as "works".

2.1 A common cause

F1, F2 and F4 share a shape: the model produced something that satisfies the *syntax* the oracle checks while defeating its *intent*. This is Goodhart's law [1] in a small, fully observable setting; in Strathern's well-known paraphrase, when a measure becomes a target, it ceases to be a good measure [2]. F2 is the purest case, since it edits the measuring instrument itself. F3 is different in kind: it is not gaming, but a policy the oracle never expressed (Debian builds must not touch the network, and everything shipped must be accounted for in `copyright` [3]).

This distinction matters for the remedy. Gaming failures are fixed by closing loopholes in the acceptance predicate, and cannot be fixed reliably by instructing the model, because an instruction raises the cost of the shortcut without removing its reward. Missing-policy failures are fixed by adding the policy, and *can* be partly addressed in the prompt, but are still best enforced mechanically.

3 A negative specification: `antilint`

Lintian encodes what Debian considers wrong with a package. The autopackager needs a second, smaller body of rules encoding what is wrong with an *autopackaged* package: the shortcuts a generator under pressure from (1) is known to take. We call this layer `antilint`. Its defining properties are:

1. It runs *outside* the model, on the artifacts and build logs, and its verdict is part of *Accept*. The model can be told about it, but cannot argue with it.

2. The model cannot write to it. Where lintian has overrides, `antilint` has none, and lintian itself is run with `--no-override`, so an override file earns nothing.

3. Every rule carries a severity, a rationale, and a *repair hint* that is returned to the model on violation, so a failed check is also useful feedback.

4. It grows from review. Every defect a reviewer finds, human or model, becomes a rule plus a regression fixture, so that no class of defect has to be found twice.

Table 1 gives an initial rule set. The first four rows are F1–F4; the rest generalise them or were already stated as prompt-level prohibitions and should move into the checker.

ID	Sev.	Rule (and mechanical test)
AL01	error	No `*.lintian-overrides` created by the generator; lintian run with `--no-override`.
AL02	error	Every patch in `series` contains at least one hunk, applies cleanly, and changes the tree; no patch file exists outside `series`.
AL03	error	The build performs no network access (build under `unshare -n` or sbuild without network; grep `rules` for fetchers as a cheap pre-check).
AL04	error	Each binary package ships at least one file outside `/usr/share/doc`; every expected artifact from the analysis record (§5) appears in some `.deb`.
AL05	error	No install or build command in `rules` is masked by `|| true`, `2>/dev/null`, or an existence guard that can skip installation.
AL06	error	No binary or large (>64 KiB) files in `debian/` (vendored blobs, jars, tarballs).
AL07	error	Every vendored directory detected by license scanning has a matching `Files:` stanza in `debian/copyright`.
AL08	warn	Upstream tests are not disabled without a recorded reason.
AL09	warn	No hard-coded build-host paths (`/home/`, VM user names) in any installed file.
AL10	warn	A revision does not delete a failing check's input (e.g. removing a test directory, emptying a manpage) to make the check pass.

Table 1: Initial `antilint` rules as proposed. The implementation (§10) splits several into sub-rules and adds AL11. "Expected artifact" is defined by the analysis record of §5.

3.1 Representation

A rule base of this kind is naturally a logic program. An extractor turns the source tree, build log and `.deb` contents into ground facts; each rule is a clause deriving `violation/4`. This keeps the rules declarative, auditable and diffable, lets a rule's rationale live beside it, and matches the Prolog-fact storage already used by `packaging-queue`. A sketch:

```
% Facts produced by the extractor, e.g.
%   debian_file(souffle, 'debian/patches/0001-....patch', 812).
%   patch_hunks(souffle, '0001-....patch', 0).
%   in_series(souffle, '0001-....patch').
%   deb_file(fast_downward, 'fast-downward', '/usr/share/doc/...').
%   expected_artifact(fast_downward, '/usr/bin/fast-downward').

violation(P, al01, error, F) :-
    debian_file(P, F, _),
```

```
    sub_atom(F, _, _, 0, 'lintian-overrides').

violation(P, al02, error, Patch) :-
    patch_file(P, Patch),
    (   patch_hunks(P, Patch, 0)
    ;   \+ in_series(P, Patch)
    ).

violation(P, al04, error, Bin) :-
    binary_package(P, Bin),
    \+ ( deb_file(P, Bin, Path),
         \+ sub_atom(Path, 0, _, _, '/usr/share/doc/') ).

violation(P, al04, error, missing(Art)) :-
    expected_artifact(P, Art),
    \+ deb_file(P, _, Art).

accept(P) :-
    builds(P), lintian_clean_no_override(P),
    \+ violation(P, _, error, _).
```

The implementation differs in detail from this sketch (facts are keyed by an attempt, and rule conditions run through a whitelisting interpreter; see §10), but keeps its shape. Note that AL04's second clause depends on `expected_artifact/2`, which no amount of inspecting the `debian/` directory can supply. That is the job of the next section.

3.2 Relation to lintian

Several of these rules overlap checks lintian already makes. A patch under `debian/patches/` that no series file lists is reported by lintian as `patch-file-present-but-not-mentioned-in-series` (a warning; compare AL02b); a series entry naming a missing patch is `quilt-series-references-non-existent-pat` (an error; AL02c); and a binary package with no contents is `empty-binary-package` (a warning), closely related to AL04a and to F4 [4]. We are not aware of a lintian check for a patch with no hunks (AL02a), for an override written by the packager as such (AL01), or for masked commands and guarded installs in `debian/rules` (AL05).

Looking at why the overlapping checks did not stop F1 and F4 exposed two properties of the pipeline rather than gaps in lintian. First, the loop accepted a package when lintian reported no *errors*, so warnings, including the two above, never blocked acceptance. Second, the loop builds binary packages only (`dpkg-buildpackage -b`), so the `.changes` file handed to lintian lists no source package and lintian's source-package checks, the patch checks among them, did not run at all. Lintian also honoured the overrides file of F2.

The gate therefore now runs lintian with `--no-override`, and treats warnings as blocking unless the tag appears on a short list of tolerated tags that is maintained by hand, kept outside anything the model writes, and requires a reason for each entry. The design rule is that `antilint` should not duplicate lintian: where lintian has a check, the remedy is to run lintian more strictly, and an `antilint` rule that fires exactly where a lintian tag fires is a candidate for removal. AL04a is such a candidate. AL02b and AL02c are not, as long as builds are binary-only, because `antilint` reads the source tree directly and is then the only check on `debian/patches/`. `antilint` also reads lintian's output as facts, so its rules can refer to lintian tags rather than re-derive them.

4 Behavioural oracles

Rules say what must be absent. They cannot say that the package works. For that the acceptance predicate should grow a small set of dynamic checks, in roughly increasing cost:

1. **Smoke test.** Install the `.deb` in a clean container and run each shipped executable with `--version` or `--help`; require exit status 0 or a recognised usage message. This alone would have caught F4 and would test the unverified claim about `ape`.
2. **Generated autopkgtest.** Have the model write `debian/tests/control` with at least one functional test (for a prover, prove a trivial theorem; for a planner, solve a toy problem) and run it with `autopkgtest` [5]. A test that the model wrote is weaker evidence than one a human wrote, so tests must themselves be screened by rules (no `exit 0`-only tests).
3. **Lifecycle.** `piuparts` install/upgrade/remove/purge cycles [6].
4. **Install-manifest diff.** Compare the `.deb` contents with what upstream's own `make install DESTDIR=...` (or equivalent) produces. Large unexplained differences in either direction are suspicious.
5. **Reproducibility** (later): `reprotest` [7].

5 Structured generation: an analysis record

The naive loop asks the model for `debian/` directly. We propose one explicit intermediate step: before writing any packaging, the model produces a typed *analysis record* for U, stating the build system and helper to use; build and runtime dependencies mapped to Debian package names; licences and vendored components; whether the build as written needs the network, and what would replace it; and the list of artifacts a correct install must contain (executables, libraries, data).

The record serves three purposes. It gives the generator a plan to condition on, which is the useful core of chain-of-thought prompting [8] without its verbosity. It is *checkable* against the build: every expected artifact must appear in some `.deb` (AL04), every vendored component must appear in `copyright` (AL07), and a record that says "needs network" forces an explicit decision rather than a silent `wget`. And it is reviewable: a human can correct a wrong record in a minute, which is far cheaper than debugging the packaging it produced.

In this light, F4 is no longer invisible. Suppose the record lists `/usr/bin/fast-downward` as an expected artifact and the `.deb` does not contain it: that is a contradiction the checker reports.

5.1 On CoT, GoT and backtracking

Graph-of-thought [10] and tree-of-thought [9] methods search over partial solutions with a scorer at each node. Their value depends entirely on the quality of that scorer, and the scorer available here is precisely (1), the thing shown to be exploitable. Adding search against a gameable verifier finds exploits faster. We therefore rank these methods below verification work, and recommend only two cheap forms:

- **Best-of-n with verifier selection.** Generate several candidate packagings (or several revisions at a hard step) in parallel and select by the strengthened *Accept* plus a size and complexity penalty. This parallelises trivially and improves automatically as the verifier improves.

- **Checkpoint rollback, not deep backtracking.** If the revision loop fails k consecutive times, or oscillates between two error sets, discard the revisions and return to the analysis record, on the theory that an early decision (wrong build helper, wrong binary split, a missing dependency) is wrong. This is one level of backtracking with a principled trigger, and it is cheap.

6 Revision discipline

Most of F1–F4 were introduced during revision, when the model is told "this failed, fix it". The revision step should be constrained accordingly:

- **Monotonicity.** A revision may not reduce the set of passing checks, and may not delete inputs to a failing check (AL10).
- **Forbidden fixes.** Overrides, no-op patches, masked commands and disabled tests are rejected mechanically, whatever the prompt says.
- **Explained edits.** Each revision states which error it addresses and why its change fixes it; the statement goes into the provenance log alongside the model and backend that produced it.
- **Budgets.** Cap diff size per revision and total revisions; a package that needs forty revisions is reported, not accepted.

7 Grading and honesty

Not every package will reach the full bar, and the unofficial archive should say so. We propose publishing a grade with each package: **C**, builds and passes `antilint`; **B**, additionally passes smoke tests and `piuparts`; **A**, additionally passes a functional autopkgtest and has had human review. The grade is cheap to compute, makes the archive honest about what it offers, and gives users and reviewers an obvious place to start.

8 Evaluation

Packages already in Debian form a natural held-out benchmark. Hide the existing `debian/` directory, autopackage the upstream source, and compare: does ours build, what does lintian say, do both ship the same files, do the dependencies agree? This measures quality against expert work rather than against our own oracle, and is the most direct defence against optimising the wrong thing. Results should be stratified by difficulty, so that trivial packages like `vampire` do not dominate the score. Other distributions' recipes (Fedora spec files, Arch `PKGBUILD`s, Nix and Guix expressions) are useful both as retrieval context for the generator and as independent evidence for the analysis record.

9 Recommended order of work

1. Run lintian with `--no-override` and add AL01–AL06 as a mechanical gate. This closes the known loopholes.
2. Add the smoke test. It is small and catches hollow packages outright.

3. Introduce the analysis record and the checks that depend on it (AL04 second clause, AL07).

4. Build the held-out benchmark from a few dozen packages already in Debian.

5. Only then: best-of-n selection and checkpoint rollback.

10 Implementation status and early results

`antilint` now exists as a separate SWI-Prolog program, deliberately kept apart from the scheduler, `packagingqueue`: the queue schedules, and `antilint` judges. An extractor turns a source tree, its `.deb` files and its logs into ground facts keyed by a subject `Pkg@Attempt`. Twenty core rules implement Table 1, with several rules split (for example AL02a–c for empty, unlisted and missing patches) and one addition, AL11, which fails a package whose install verification failed. Rule conditions are evaluated by a small whitelisting interpreter rather than called directly.

Belief in the rules themselves is maintained as a justification-based truth maintenance problem. Every snapshot, human verdict, finding, endorsement and waiver is an append-only event, and a rule's status is derived from them on each query, never stored; a command prints the grounds for any rule's status. A core rule that fires on a package judged good is marked *disputed* for a human to examine. Learned rules are instances of a few fixed schemas, never arbitrary code; evidence alone can raise one to a warning, only a named human's endorsement lets it fail a build, and a single counterexample from the known-good corpus demotes it. The fact base is not under truth maintenance, since it is recomputed from the artifacts on every run.

A first session on real artifacts gave the following.

F2 is caught.
Checking the generated `val` tree reports AL01 on `debian/val.lintian-overrides` and exits non-zero, which would fail that iteration.

The gold corpus checks the checker.
Debian's own packaging of `eprover` was snapshotted as known-good. Two core rules immediately fired on it, AL01 and AL07a, and were marked disputed. The AL01 case is instructive: Debian maintainers do use lintian overrides, with reasons, so an override is not wrong in itself; what AL01 encodes is distrust of the *generator*. Such rules are now declared generator-only, so a human-maintained gold package is not a counterexample to them, while a generated package that a human judges good still is. The AL07a hit is still being examined.

Blunt learned rules are demoted.
A finding on `fast-downward` was turned into a learned rule forbidding the text `test -f` in `debian/rules`. The gold corpus suspended it at once, because `eprover`'s packaging uses `test -f` legitimately. The surface pattern was the wrong generalisation of F4.

A fifth defect.
While preparing the `val` test, the same model-assisted review found that its upstream tree bundles an `externals/` directory, including a Windows `.dll`, with no corresponding stanza in `debian/copyright`. This is a failure of the F3 kind, and a case AL07 is meant to cover.

The known-bad verdicts above rest on the model-assisted review of Section 2, accepted by the author; an independent review by a Debian developer is the next check on them, and on the rules derived from them.

The main lesson so far is that the known-good corpus is the most valuable part of the design: within minutes it found errors in rules that had looked obviously right, including one taken directly from a reviewed defect. Of the order of work in the previous section, step 1 is implemented but not yet wired into the autopackager loop.

11 Conclusion

The naive autopackager's oracle was the right place to start and the wrong place to stop. Its failures are not random; they are the shortcuts the oracle rewards. The remedy is mostly on the verification side: a negative specification the model cannot edit, behavioural checks that test what a package does rather than how it looks, and an analysis record that makes the generator's claims checkable. Smarter search over generations becomes worthwhile once there is something trustworthy to search against.

References

[1] C. A. E. Goodhart. Problems of monetary management: the U.K. experience. In *Papers in Monetary Economics*, Reserve Bank of Australia, 1975.

[2] M. Strathern. 'Improving ratings': audit in the British University system. *European Review*, 5(3):305–321, 1997.

[3] Debian Policy Manual, §4.9 ("Main building script: debian/rules") and §12.5 ("Copyright information"). `https://www.debian.org/doc/debian-policy/`

[4] Lintian tag descriptions: `patch-file-present-but-not-mentioned-in-series`, `quilt-series-references-non-existent-patch`, `empty-binary-package`. `https://lintian.debian.org/tags/`

[5] autopkgtest: automatic as-installed testing for Debian packages. `https://salsa.debian.org/ci-team/autopkgtest`

[6] piuparts: .deb package installation, upgrading, and removal testing tool. `https://piuparts.debian.org/`

[7] reprotest: build packages and check them for reproducibility. `https://salsa.debian.org/reproducible-builds/reprotest`

[8] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[9] S. Yao et al. Tree of thoughts: deliberate problem solving with large language models. In *NeurIPS*, 2023.

[10] M. Besta et al. Graph of thoughts: solving elaborate problems with large language models. In *AAAI*, 2024.